

Datathon Master Revision Notes: Module 3

Deep Statistical Intuition & Mathematical Derivations

Roman

May 21, 2026

The Z-Score Standardization System

1. The Core Engineering Logic

In a raw datathon dataset, features live on completely different scales. For example, **Age** ranges from 18 to 80, while **Annual_Income** ranges from \$20,000 to \$500,000. If these features are fed directly into distance-based models (like K-Means or KNN) or gradient-driven models (like Linear Regression or Neural Networks), the massive numeric scales of income will completely drown out the age feature. The Z-score standardization transforms every continuous column into a unitless metric space centered at a mean (μ) of 0 with a spread/standard deviation (σ) of 1.

2. Mathematical Derivation

Let x be an individual raw feature observation, μ be the population mean of that feature column, and σ be the standard deviation (the average variance distance). The standardization operation executes a two-step transformation:

1. **Mean Centering:** Shift the entire distribution so the average becomes zero: $x_{\text{centered}} = x - \mu$
2. **Variance Scaling:** Divide the centered distance by the standard deviation unit yardstick to strip away physical units:

$$z = \frac{x - \mu}{\sigma}$$

3. Concrete Datathon Example

Consider a continuous feature tracking **Session_Length** on an app. Across the training set, the mean length $\mu = 300$ seconds and the standard deviation $\sigma = 50$ seconds.

- **User A** logs a session of 300 seconds: $z = \frac{300-300}{50} = 0$. They are exactly average.
- **User B** logs a session of 450 seconds: $z = \frac{450-300}{50} = \frac{150}{50} = +3.0$. This user is exactly 3 standard deviations above the average.

4. Geometric Visualization Profile

When you apply this formula to a continuous Gaussian distribution, it maps perfectly onto the ****Standard Normal Curve****.

- **The Interval $[-1, +1]$:** Contains approximately **68%** of all user behaviors.
- **The Interval $[-2, +2]$:** Contains approximately **95%** of all ordinary data points.

- **Beyond ± 3 :** Contains only **0.3%** of the data. If your preprocessing script encounters a row where $z = +4.5$, your structural intuition flags this as an extreme outlier or an anomalous session event.

The Parameter Engine: Likelihood vs. Log-Likelihood (MLE)

1. The Core Engineering Logic

When training a classification layer (like Logistic Regression), the model looks at a series of binary targets ($y = 1$ for click, $y = 0$ for no-click). We assume this behavior follows a Bernoulli distribution governed by an unknown probability parameter p . Maximum Likelihood Estimation (MLE) answers the foundational question: *What exact parameter value for p maximizes the probability that the specific combination of zeros and ones on our screen actually occurred?*

2. Mathematical Derivation from First Principles

Let our observed sample data points be $x_1, x_2, \dots, x_n$ where $x_i \in \{0, 1\}$. The probability mass function for a single independent Bernoulli trial is written as:

$$P(x_i|p) = p^{x_i}(1-p)^{1-x_i}$$

Because each user behavior is independent, the joint probability of observing the entire dataset simultaneously is the product of their individual probabilities. This defines the **Likelihood Function**, $L(p)$:

$$L(p) = \prod_{i=1}^n p^{x_i}(1-p)^{1-x_i}$$

The Optimization Bottleneck: Multiplying thousands of decimal probabilities together causes **numerical underflow**—the computer's floating-point register runs out of memory slots, rounds the product to absolute 0, and crashes the gradient calculations.

To resolve this, we apply the natural logarithm ($\ln$). Because logarithms are monotonically increasing, the parameter value that maximizes the log function is identical to the value that maximizes the raw function. The log operator transforms all internal multiplications into clean additions, generating the **Log-Likelihood Function**, $\ln L(p)$:

$$\ln L(p) = \ln \left(\prod_{i=1}^n p^{x_i}(1-p)^{1-x_i} \right) = \sum_{i=1}^n \ln (p^{x_i}(1-p)^{1-x_i})$$

$$\ln L(p) = \sum_{i=1}^n [x_i \ln(p) + (1-x_i) \ln(1-p)]$$

To find the absolute peak of this likelihood curve, we use calculus. We take the partial derivative with respect to our parameter p , set the equation equal to 0, and solve for p :

$$\frac{d}{dp} \ln L(p) = \sum_{i=1}^n \left[\frac{x_i}{p} - \frac{1-x_i}{1-p} \right] = 0$$

$$\sum_{i=1}^n \frac{x_i(1-p) - p(1-x_i)}{p(1-p)} = 0 \implies \sum_{i=1}^n (x_i - x_i p - p + x_i p) = 0$$

$$\sum_{i=1}^n x_i - \sum_{i=1}^n p = 0 \implies \sum_{i=1}^n x_i - np = 0 \implies p_{\text{MLE}} = \frac{1}{n} \sum_{i=1}^n x_i$$

The Structural Insight: The calculus derivative proves that the mathematically optimal parameter choice for a Bernoulli distribution is simply the sample mean of your data columns!

Furthermore, look closely at the structure of $\ln L(p)$. If you multiply it by -1 , it becomes the exact mathematical formula for **Binary Cross-Entropy Loss** used by neural networks and XGBoost. Minimizing classification loss is structurally identical to maximizing the log-likelihood of reality.